Detecção de Falhas por Heartbeat e Eleição de Líder por *Bully*: Implementação e Avaliação Experimental

Guilherme Webster Chamoun (RA 257111)
Instituto de Computação
Universidade Estadual de Campinas
g257111@dac.unicamp.br

Gabriel Cunha Marchetti (RA 251055)
Instituto de Computação
Universidade Estadual de Campinas
g251055@dac.unicamp.br

Repositório: Link GitHub | **Video:** Link Vídeo

***Resumo*—Foram implementados dois algoritmos distribuídos clássicos que se complementam para criar um sistema distribuído: um detector de falhas por *heartbeat*, com *timeout* fixo ou adaptativo, e a eleição de líder pelo algoritmo do *Bully*, disparada quando o detector suspeita do líder. Cada nó é um contêiner independente que se comunica apenas por datagramas UDP. A correção é verificada por testes determinísticos em um simulador e por verificadores reprodutíveis sobre os logs. Em 78 ensaios com Docker, todos os sobreviventes convergiram para o maior ID ativo. Com resposta configurada em 500 ms, a eleição levou em média entre 501,3 e 502,0 ms para $N \in \{3, 5, 7, 9, 11, 13\}$. O tempo médio entre o comando de falha e o acordo ficou entre 1,004 e 1,065 s. Em 80 ensaios simulados controlados, o custo foi N mensagens no melhor caso e $N(N-1)$ no pior, para $N \in \{3, 5, 7, 9\}$.**

I. Introdução

Em vários contextos, é importante um algoritmo que escolhe um nó que é de certa forma "especial". No contexto de sistemas *peer-to-peer* (P2P), por exemplo, é conveniente um super-*peer* para evitar comunicação excessiva, uma melhor garantia nos recursos disponíveis para atender demais nós e entre outras razões. De todo modo, outro aspecto importante é a dinâmica do sistema: e se um super-*peer* sair ou falhar? Isto precisa ser detectado de maneira a eleger um novo, sendo assim uma necessidade importante, é saber distinguir um nó morto de um nó lento (dentro do aceitável) e a concordância entre os nós de quem será o novo líder para gerar estados consistentes do sistema.

Neste trabalho, implementamos um algoritmo para cada um dos dois problemas apresentados. Detecção de falhas por *heartbeat* e eleição de líder por *bully*; os dois se integram em um serviço com líder tolerante a falhas: o detector observa o líder e, ao suspeitar dele, dispara uma eleição.

As contribuições são: (i) a implementação dos dois algoritmos a partir de um transporte UDP próprio, sem bibliotecas que já os implementem; (ii) um simulador determinístico de rede e relógio, usado para testar propriedades de correção sob perda, partição e queda; (iii) verificadores de propriedades sobre os logs das execuções reais; e (iv) uma avaliação experimental automatizada e reproduzível, que relaciona os tempos medidos aos *timeouts* e o número de mensagens à complexidade do algoritmo.

II. Modelo do Sistema

A. Detector de falhas por heartbeat

Adotamos sincronia parcial: existem limites para o atraso das mensagens e para a velocidade relativa dos processos, mas eles são desconhecidos ou só valem após um instante de estabilização (GST - *Global Stabilization Time*). Assim, o detector não é perfeito; ele pode suspeitar erradamente de um nó lento. A precisão depende, então, da configuração.

Com *timeout* fixo T e intervalo Δ, não há falsas suspeitas enquanto nenhum atraso exceder $T - \Delta$. Com perdas, falsas suspeitas acontecem com frequência, mas cada uma é desfeita pelo próximo *heartbeat*. Com *timeout* adaptativo sem teto, cada falsa suspeita aumenta o *timeout* daquele par; depois do GST, isso ocorre algumas vezes (dada uma comunicação estável) e processamento, e uma hora a convergência é retomada.

Não há relógio sincronizado: cada nó mede *timeouts* com seu próprio clock. Um nó que reinicia volta sem estado e com uma nova *encarnação* (o instante de início em ms). Assim, os demais conseguem distinguir um reinício de uma falsa suspeita.

B. Eleição Bully

O algoritmo do *Bully* supõe um sistema síncrono: um *timeout* sem resposta significa que o destinatário falhou. Quando ela é violada (perda alta, nó pausado, particionamento, etc), o algoritmo pode eleger líderes diferentes em lados distintos (*split-brain*). A propriedade de segurança é, portanto, *apos a estabilização*, e não em todo instante. Durante a eleição, nós diferentes podem conhecer líderes diferentes ou nenhum. Nenhum estado é persistido; o nó que reinicia convoca uma eleição e, se tiver o maior ID, retoma a lideranca.

III. Arquitetura

Cada nó lógico é um conteiner independente. O arquivo `compose.yaml` é gerado para qualquer N por um script. Os nós só se comunicam pela rede do Compose, e toda a configuração (identificador, *peers*, *timeouts*, injeção de falhas, semente) vem de variáveis de ambiente. Assim, o mesmo código pode rodar em outro ambiente alterando apenas a configuração. A Fig. 1 mostra os componentes de um nó.

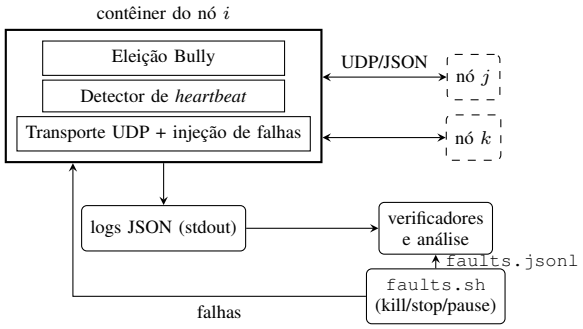


Figura 1. Arquitetura de um nó. O detector notifica o Bully por *callbacks*; os logs e o registro de falhas alimentam os verificadores e a análise.

A. Transporte e mensagens

As mensagens são JSON's UDP, com algum dos tipos (`HEARTBEAT`, `ELECTION`, `ANSWER`, `COORDINATOR`), remetente, destinatário, identificador único e carga útil. Escolhemos UDP porque *heartbeats* são naturalmente datagramas desse estilo e porque o canal fica explicitamente sujeito a perdas, duplicação e reordenação: os algoritmos tratam essas falhas em vez de depender de garantias do TCP. O envio é *best-effort* e nunca bloqueia. Erros de rede viram eventos `msg_drop`. Cada algoritmo registra um *handler* por tipo de mensagem.

B. Injeção de falhas

Usamos dois níveis de falhas. No nível da aplicação, o transporte descarta mensagens com probabilidade `DROP_RATE`, atrasa-as por `DELAY_MS` mais um valor uniforme em $[0,$ `JITTER_MS`$]$ e pode bloquear destinos , isto ajuda a ver casos em que o sistema cai em *split-brain*. Os sorteios usam um gerador semeado por `SEED` e pelo ID do nó. No nível do contêiner, o script `faults.sh` executa `docker kill` (*crash-stop*), `stop`/`start` (*crash-recovery*) e `pause`/`unpause` (nó lento) e registra em `faults.jsonl` o instante em que cada comando foi disparado e concluído. Preferimos esses mecanismos ao `tc`/`netem` porque são reprodutíveis por semente e não exigem privilégios (`NET_ADMIN`) nos contêineres.

C. Observabilidade

Cada nó emite um evento JSON por linha (`suspect`, `restore`, `election_start`, `leader_change`, `msg_send`, entre outros), com *timestamp*. Os logs servem, ao mesmo tempo, de evidência de correção e de fonte para as métricas. Como todos os contêineres são executados em uma mesma máquina, os *timestamps* são comparáveis entre nós; em máquinas distintas, isso exigiria sincronização.

IV. Algoritmos

A. Detector de falhas por heartbeat

A cada Δ = `HB_INTERVAL_MS`, cada nó envia um sinal de *heartbeat* a todos os demais com nós com a devida identificação. Para cada *peer*, o receptor mantém um temporizador de duração T = `HB_TIMEOUT_MS`, reiniciado a cada *heartbeat* novo.

Quando o temporizador expira, o *peer* passa a ser suspeito (`suspect`). Um *heartbeat* de um suspeito o restaura (`restore`). Se a encarnação (um identificador de instanciação) for a mesma, a suspeita era falsa; se for nova, o nó reiniciou.

No modo adaptativo, cada falsa suspeita soma `HB_TIMEOUT_STEP_MS` (200 ms) ao timeout daquele *peer*, até um teto opcional. Assim, os pares mais "atrasados"passam a ser vigiados com maneira mais branda.

B. Eleição Bully

Um nó inicia uma eleição ao subir, ao suspeitar do líder ou ao ver voltar um nó de ID maior que o lider corrente. Ele envia `ELECTION` a todos os nós de ID maior, inclusive aos suspeitos, já que suspeita não é, ainda, falha. Em seguida, espera a mensagem `ANSWER` por até `ELECTION_TIMEOUT_MS`. Sem resposta, declara-se líder e envia `COORDINATOR` a todos. Com resposta, espera o `COORDINATOR` por até `COORD_TIMEOUT_MS` e, se ele não chegar, reinicia a eleição. Quem recebe `ELECTION` de um nó menor responde `ANSWER` e inicia a própria eleição; se já for o líder, reenvia o anúncio, o que recupera um `COORDINATOR` perdido.

Cada tentativa tem um `election_id`, usado para descartar respostas antigas e correlacionar eventos. Um nó aceita apenas uma tentativa por vez: pedidos recebidos durante uma eleicao não criam outras tentativas paralelas; anúncios de um coordenador menor são contestados com outra eleição, e um anúncio com *delay* não substitui um líder maior que ainda não é tido como suspeito pelos seus colegas.

Reiterando, se houver uma "barreira", por exemploo, de modo que o nó 1 não veja os demais, num cenário de 4 nós, ele irá se autodeclarar líder pois vai entender que os nós maiores falharam e isto só será resolvido quando a comunicação dele com os demais for reestabelecida. Vale notar que, ainda que o *split brain* tenha sido resolvido, as possíveis mudanças que o nó 1 fez enquanto se achava líder não são concretizadas, pois a mensagem `COORDINATOR` do nó 1 irá atingir um nó que conhece um líder de maior ID.

C. Integração

O Bully se inscreve no detector. Uma suspeita sobre o líder dispara `start_election("leader_suspected")`, e a restauração de um nó maior que o líder dispara `start_election("peer_restored")`. O detector modulariza erros e inscritos, de modo que uma falha no Bully não interrompe a detecção.

D. Complexidade de mensagens

O detector *all-to-all* envia $N(N-1)$ mensagens por intervalo, ou $N(N-1)/\Delta$ mensagens por segundo: custo $O(N^2)$. No Bully, contando todos os envios, inclusive ao líder morto, o melhor caso envia uma `ELECTION` e $N-1$ mensagens `COORDINATOR` para os nós menores no caso de iniciar uma eleição no nó de maior ID, totalizando N: $O(N)$. Quando

Tabela I
LATÊNCIA ESTIMADA DE DETECÇÃO (MS); 80 OBSERVAÇÕES POR TIMEOUT, PROVENIENTES DE DEZ SEMENTES.

T	Média	DP	p50	p95	p99
2Δ	316,5	51,3	325,7	389,0	401,7
3Δ	534,6	53,1	536,6	600,9	601,4
5Δ	927,1	56,1	925,5	1000,6	1000,8

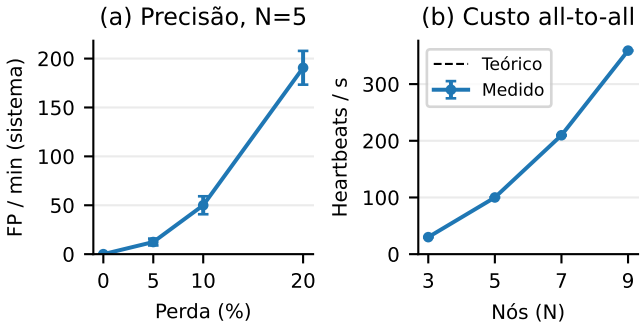


Figura 2. Heartbeat: (a) falsas suspeitas por minuto do sistema, $N = 5$; (b) mensagens/s comparadas a $N(N-1)/\Delta$. Pontos e barras representam média e ± 1 DP entre dez sementes.

o menor ID inicia a eleição e ela se propaga aos maiores, são $N(N-1)/2$ mensagens `ELECTION`, $(N-1)(N-2)/2$ respostas e $N-1$ anúncios: total $N(N-1)$, ou $O(N^2)$. O detector pode disparar eleições em vários nós; por isso, a comparação isolada dos casos usa notificações controladas no simulador (Seção VI-C).

V. CORREÇÃO E TESTES

A. Testes e verificação dos logs

Os 196 testes de `make test` passaram (em aproximadamente 5,66 s). Transporte em memória, relógio virtual e sementes fixas verificam suspeita/restauração, quedas, recuperação, duplicação e eleições. Os nove testes Docker também passaram, mas devem ser executados de forma autônoma `make test-e2e`.

O `check_heartbeat.py` cruza logs e falhas: exige detecção até $T+500$ ms, nenhuma restauração durante a queda e restauração após o retorno. Os novos ensaios verificaram 240 detecções e 240 restaurações, sem violações. Nos 78 ensaios anteriores de failover, o Bully apresentou acordo inicial e final, sem reinícios dos sobreviventes ou erros de handler.

B. Verificação experimental do heartbeat

Usamos $\Delta = 200$ ms, detector fixo, dez sementes por configuração e 5 s de aquecimento. Para $N = 5$ e $T \in \{2,3,5\} \times \Delta$, cada ensaio observa duas quedas/retornos em 28 s: 80 detecções por timeout (Tabela I). A latência é estimada desde o último sinal de vida da vítima durante o comando Docker; o crash exato permanece incerto. As 80 observações compartilham dez execuções. Percentis usam interpolação linear; DP é amostral.

Tabela II
AMBIENTE DE EXECUÇÃO DOS EXPERIMENTOS.

Item	Valor
CPU	Intel Core i5-13420H (12 CPUs lógicas)
Memória	15,32 GiB
Sistema operacional	Arch Linux, x86_64
Kernel	6.19.9-arch1-1
Docker / Compose	29.3.1 / 5.1.1
Python (imagem)	3.12.16

Para precisão, reanalisamos eventos `suspect` dos 40 ensaios sem crashes de `bully-loss`: $N = 5$, $T = 3\Delta$, observação de 30 s, excluindo aquecimento. Todos são falsos positivos: médias de 0,0; 12,4; 50,0; 190,6 por minuto do sistema para perdas de 0%, 5%, 10%, 20%.

Na escala, 40 novos ensaios mediram apenas envios `HEARTBEAT`, durante 12 s sem falhas ou perdas. Para $N \in \{3,5,7,9\}$, as médias foram 30,0; 100,0; 209,7; 359,1 mensagens/s, próximas a $N(N-1)/\Delta$ (Fig. 2). Os resultados mostram detecção mais lenta com timeout maior, precisão menor sob perdas e custo quadrático; limitam-se ao sistema integrado em uma máquina, sem avaliar adaptação. Veja que esse resultado é esperado.

VI. METODOLOGIA EXPERIMENTAL

A. Ambiente

Os três experimentos de tempos com Docker têm o mesmo hardware e versões registrados em seus arquivos `env.json` (Tabela II); portanto, essa descrição se aplica aos 78 ensaios disponíveis, sem inferir o ambiente de outros experimentos. A simulação de mensagens registra Python 3.12.16 e kernel 6.19.9-arch1-1, com alterações locais. Os tempos da simulação são virtuais. A varredura de perdas registra CPU 13th Gen Intel(R) Core(TM) i5-13420H, 15.32 GiB de RAM, kernel 6.19.9-arch1-1, Docker 29.3.1, Compose 5.1.1.

B. Métricas

Sejam t_f o instante anterior ao comando Docker de falha, t_e o primeiro evento `election_start` após esse comando e t_a o início do último período de acordo em $N-1$ que persiste até o fim da observação. Medimos $L_e = t_a - t_e$ (eleição), $L_f = t_a - t_f$ (*failover*) e $L_d = t_e - t_f$ (comando até início da eleição). Essa decomposição identifica qual espera domina a recuperação: $L_f = L_d + L_e$. Como t_f não é o instante exato do *crash*, L_d inclui o custo de execução do comando e não é uma latência pura do detector, mas é difícil de detectar esse momento.

O custo de mensagens soma todos os envios `ELECTION`, `ANSWER` e `COORDINATOR` após a queda, inclusive ao nó morto; exclui *heartbeats* e inicialização. Uma eleição corresponde aqui à substituição completa do líder, reunindo as tentativas locais dos diferentes candidatos. Reportamos média, desvio-padrão amostral (DP), mediana e percentis 95 e 99. Nos ensaios Docker, os percentis usam interpolação linear como dito anteriormente.

Tabela III
EXPERIMENTOS: PARÂMETRO VARIADO, NÍVEIS, VALORES FIXOS E PROTOCOLO.

Experimento	Métricas	Variado	Níveis	Rep.	Aquec. (s)	Obs. (s)	Fixos (ms)
Bully: escala	L_e, L_f, L_d	N	3, 5, 7, 9, 11, 13	10	10	8	$\Delta = 200, T = 600, E = 500, C = 1000$
Bully: resposta	L_e, L_f, L_d	E (ms)	200, 500, 1000	3	3	5	$N = 5, \Delta = 200, T = 600, C = 3000$
Bully: detecção	L_e, L_f, L_d	T (ms)	400, 600, 1200	3	3	5	$N = 5, \Delta = 200, E = 500, C = 3000$
Bully: mensagens (sim.)	envios por eleição	N, iniciador	3, 5, 7, 9; $N-1$ ou 1	10	10	8	$\Delta = 200, T = 600, E = 500, C = 1000$
Bully: perdas	tentativas espúrias	perda	0%, 5%, 10%, 20%	10	5	30	$N = 5, \Delta = 200, T = 600, E = 500, C = 1000$

Δ: intervalo de heartbeat; T: timeout do detector; E: espera por ANSWER; C: espera por COORDINATOR. Aquecimento e observação da simulação são tempos virtuais.

C. Experimentos e parâmetros

A Tabela III resume os conjuntos concluídos. Nos ensaios Docker de tempos não há perda ou atraso artificial e o detector usa *timeout* fixo. O líder N é morto 1 s após o aquecimento na varredura de N, e 0,5 s após o aquecimento nas varreduras de *timeout*. Os ensaios variam uma dimensão por vez e usam sementes registradas.

Na contagem simulada, a rede tem atraso fixo de 5 ms, sem perda ou *jitter*. Após 10 s virtuais de aquecimento, N cai e a observação continua por 8 s. Todos os detectores registram a suspeita normalmente, mas um adaptador notifica imediatamente o Bully apenas em $N-1$ (melhor caso) ou em 1 (pior). Os demais callbacks são atrasados em 2 s, permitindo que a eleição se propague por `ELECTION`. Manter a suspeita interna nos demais detectores é necessário para aceitar um coordenador menor que o líder anterior. O código dos algoritmos permanece inalterado. As dez sementes verificam reprodutibilidade: sem perda ou *jitter*, não introduzem variabilidade de rede.

D. Reprodutibilidade

Para os ensaios Docker: `make experiments EXP=bully-election`, `EXP=bully-timeout-election` e `EXP=bully-timeout-heartbeat`. Para eleições espúrias: `make experiments EXP=bully-loss`. A contagem controlada é reproduzida por `python scripts/run_bully_messages.py`. Os diretórios `experiments/raw/` guardam logs comprimidos, configuração, sementes, identificação do código e ambiente; `experiments/results/` contém os CSVs por ensaio e os agregados. As tabelas deste relatório usam os arquivos `summary.csv`. Os gráficos de tempos foram produzidos pelos scripts em `notebook/`.

VII. RESULTADOS E DISCUSSÃO

A. Detector de falhas por heartbeat

Os resultados disponíveis permitem avaliar a etapa anterior à eleição no experimento integrado, com $N = 5$ e $\Delta = 200$ ms. Ao elevar o *timeout* do detector de 400 para 600 e 1200 ms, o tempo médio do comando de falha até o início da eleição passou de 0,4291 para 0,5285 e 1,1505 s (Tabela IV). O aumento é compatível com maior espera para suspeitar do líder. O custo do comando Docker e a fase dos *heartbeats* contribuem para a variação; esses valores não devem ser comparados diretamente a um limite de detecção contado a partir do instante exato do *crash*.

Tabela IV
COMANDO DE FALHA ATÉ INÍCIO DA ELEIÇÃO, L_d (S), POR TIMEOUT DO DETECTOR T (MS). DADOS DO EXPERIMENTO INTEGRADO, $N = 5$.

Config.	n	Média	DP	p50	p95	p99
$T = 400$	3	0,4291	0,0903	0,4184	0,5137	0,5222
$T = 600$	3	0,5285	0,0132	0,5270	0,5409	0,5422
$T = 1200$	3	1,1505	0,0455	1,1517	1,1910	1,1945

Tabela V
TEMPOS DO BULLY (S) POR NÚMERO INICIAL DE NÓS. DEZ ENSAIOS POR CONFIGURAÇÃO; PERCENTIS COM INTERPOLAÇÃO LINEAR.

Config.	n	Média	DP	p50	p95	p99
Início da eleição até acordo (L_e)						
$N = 3$	10	0,5013	0,0006	0,5011	0,5022	0,5026
$N = 5$	10	0,5013	0,0005	0,5014	0,5020	0,5021
$N = 7$	10	0,5014	0,0004	0,5013	0,5019	0,5021
$N = 9$	10	0,5018	0,0004	0,5018	0,5023	0,5025
$N = 11$	10	0,5017	0,0003	0,5016	0,5022	0,5023
$N = 13$	10	0,5020	0,0005	0,5019	0,5027	0,5030
Comando de falha até acordo (L_f)						
$N = 3$	10	1,0039	0,0635	0,9804	1,1073	1,1666
$N = 5$	10	1,0646	0,0957	1,0432	1,1815	1,1879
$N = 7$	10	1,0418	0,0546	1,0308	1,1309	1,1342
$N = 9$	10	1,0567	0,0804	1,0127	1,1865	1,2153
$N = 11$	10	1,0556	0,0582	1,0389	1,1405	1,1452
$N = 13$	10	1,0637	0,0622	1,0561	1,1475	1,1555

B. Eleição Bully

A Fig. 3 e a Tabela V mostram os 60 ensaios de variação de N. Com espera por resposta de 500 ms, as médias da eleição ficaram entre 501,3 e 502,0 ms, e o DP entre 0,34 e 0,55 ms. O pequeno aumento de tempo com o aumento de N é muito inferior ao tempo de espera configurado. Nessas condições, a eleição é dominada pelo *timeout* ($T_{\text{comunicação/processamento}} \approx 0$):

$$L_e \approx E + T_{\text{comunicação/processamento}}. \tag{1}$$

O *failover* médio variou entre 1,004 e 1,065 s, com DP de 54,6 a 95,7 ms. A parcela anterior à eleição explica a maior parte dessa variabilidade; o número de nós, isoladamente, não produziu crescimento monotônico no *failover*, acreditamos que pelo mesmo processo da eleição.

A varredura exploratória dos *timeouts*, com três ensaios por configuração, confirma a separação entre as duas esperas (Fig. 4 e Tabela VI). Ao variar E de 200 para 1000 ms, a

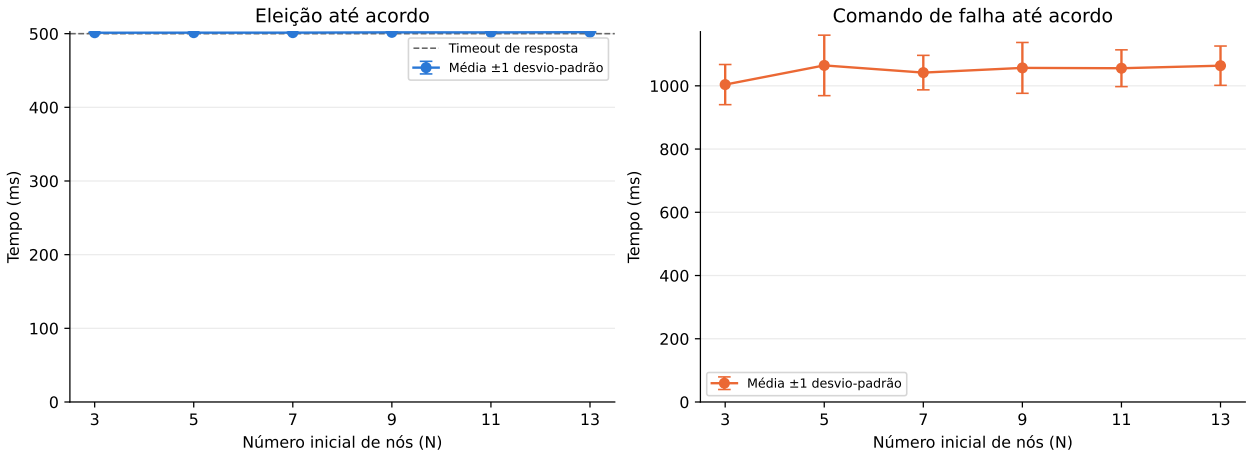


Figura 3. Tempos de eleição e de failover por número inicial de nós, em Docker. Cada ponto resume dez ensaios; barras indicam ±1 DP amostral. A referência de 500 ms corresponde à espera por ANSWER. Os painéis usam escalas verticais próprias.

Tabela VI
EFEITO DOS TIMEOUTS (MS) NOS TEMPOS (S), $N = 5$. TRÊS REPETIÇÕES POR CONFIGURAÇÃO.

		Eleição (L_e)		Failover (L_f)	
Variado	n	**Média**	**DP**	**Média**	**DP**
$E = 200$	3	0,2016	0,0004	0,7631	0,0248
$E = 500$	3	0,5017	0,0006	1,0639	0,0455
$E = 1000$	3	1,0012	0,0005	1,5627	0,0194
$T = 400$	3	0,5015	0,0004	0,9305	0,0906
$T = 600$	3	0,5008	0,0001	1,0294	0,0133
$T = 1200$	3	0,5015	0,0002	1,6520	0,0453

Tabela VII
MENSAGENS ENVIADAS NO SIMULADOR. E/A/C: ELECTION, ANSWER E COORDINATOR. VALORES IDÊNTICOS NAS DEZ REPETIÇÕES DE CADA CASO.

	Melhor: detector $N-1$		Pior: detector 1	
N	E/A/C	Total	E/A/C	Total
3	1/0/2	3	3/1/2	6
5	1/0/4	5	10/6/4	20
7	1/0/6	7	21/15/6	42
9	1/0/8	9	36/28/8	72

eleição média aumenta de 0,2016 para 1,0012 s, indicando um valor baixo de tempo de processamento e comunicação, e o *failover* de 0,7631 para 1,5627 s. Ao variar T (*timeout* do detector de falhas por *heartbeat*) de 400 para 1200 ms, mantendo $E = 500$ ms, a eleição permanece próxima de 0,501 s, enquanto o *failover* sobe de 0,9305 para 1,6520 s. Não avaliamos aqui a variação do tempo de espera por COORDINATOR. Veja que essa observação está em de acordo com as métricas estabelecidas em (Seção VI-B).

A comparação controlada de mensagens no simulador confirma os dois regimes previstos (Fig. 5 e Tabela VII). O melhor caso produziu 3, 5, 7 e 9 envios; o pior, 6, 20, 42 e 72. Todos os 80 ensaios terminaram em acordo em $N - 1$, sem erros de *handler*. As dez repetições de cada configuração tiveram contagens idênticas: DP zero e p50 = p95 = p99 = média, como esperado sem perda ou *jitter*. Isso verifica reprodutibilidade no simulador, sem estimar a variabilidade do escalonamento real.

1) Eleições espúrias sob perda: Foram executados 40 ensaios adicionais em Docker, com $N = 5$, dez sementes por taxa de perda, 5 s de aquecimento e 30 s de observação. Mantivemos $\Delta = 200$ ms, $T = 600$ ms, $E = 500$ ms e $C = 1000$ ms, sem adaptação do detector. Não foram injetados crashes, pausas ou reinícios: a perda de 0%, 5%, 10% ou 20% foi aplicada a todos os tipos de mensagem. Todos os ensaios preservaram os cinco nós em operação, sem erros de handler ou violações do protocolo experimental.

Contamos cada par (nó, election_id) com evento `election_start` na janela de observação, excluindo startup. São tentativas locais espúrias do sistema, não rodadas globais independentes. Uma falsa suspeita pode gerar tentativas em vários candidatos e novas tentativas ao receber um anúncio de coordenador ainda suspeito. Por isso, a contagem não equivale ao número de falsas suspeitas e pode apresentar rajadas.

Com 20% de perda, sete dos dez ensaios estavam em acordo no início da janela e nove estavam em acordo ao final. Nas demais taxas, todos estavam em acordo nesses dois instantes. A medição não exige acordo inicial nem final: a instabilidade sob perda contínua é parte do comportamento observado, mesmo sem falhas reais dos nós. Portanto, podemos observar uma situação de adversidade para a arquitetura utilizada em sistemas de alta taxa de perda.

A Tabela VIII registra as médias de tentativas por ensaio (0%: 0,0, 5%: 231,5, 10%: 1394,9, 20%: 7943,5). Normalizadas pela duração efetivamente observada, as médias são 0,0, 463,0, 2789,8, 15886,9 tentativas por minuto do sistema, respectivamente. A dispersão entre sementes deve ser considerada junto da média. A perda afeta tanto os heartbeats, que originam suspeitas, quanto ANSWER e COORDINATOR, que influenciam repetições e propagação; portanto, não se deve supor uma relação linear entre perda e contagem. Os resultados

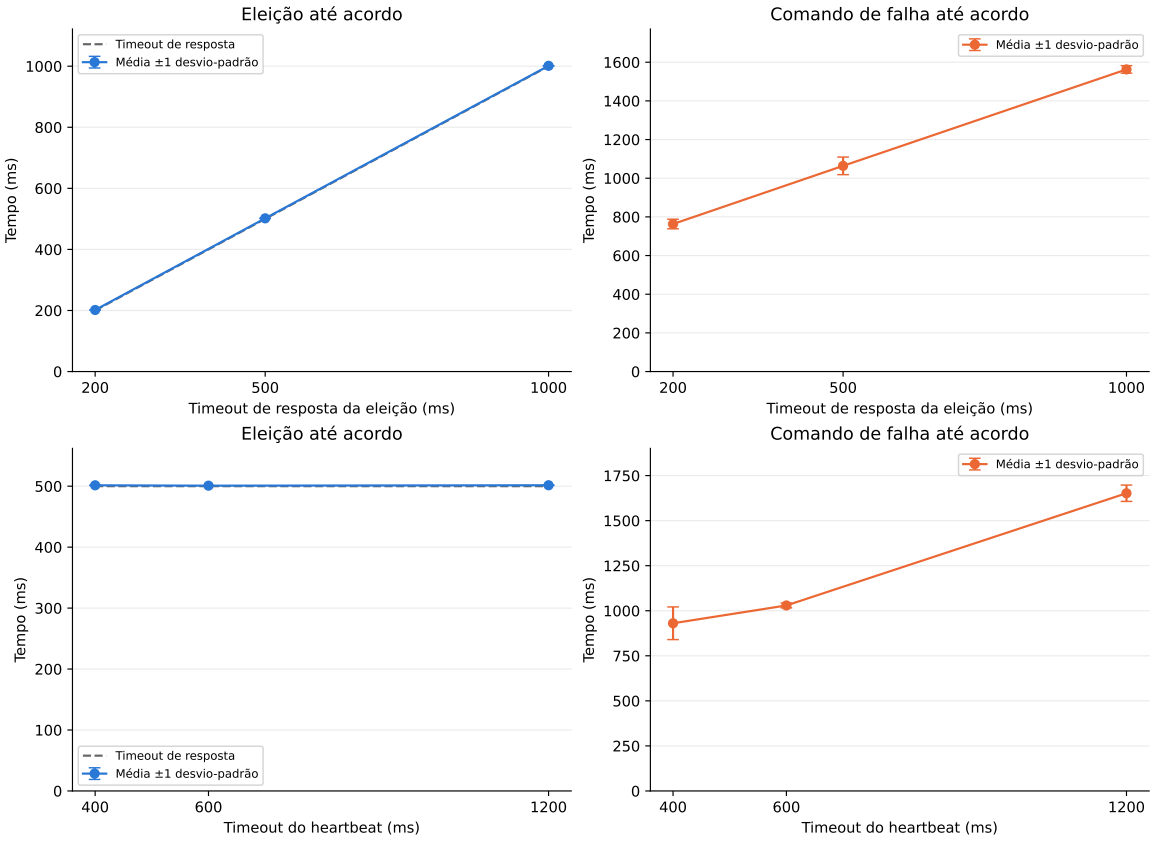


Figura 4. Efeito dos timeouts com $N = 5$, em Docker. Linha superior: variação de E (ANSWER); inferior: variação de T (heartbeat). Colunas: eleição e failover. Cada ponto representa três ensaios; barras mostram ± 1 DP, não intervalos de confiança.

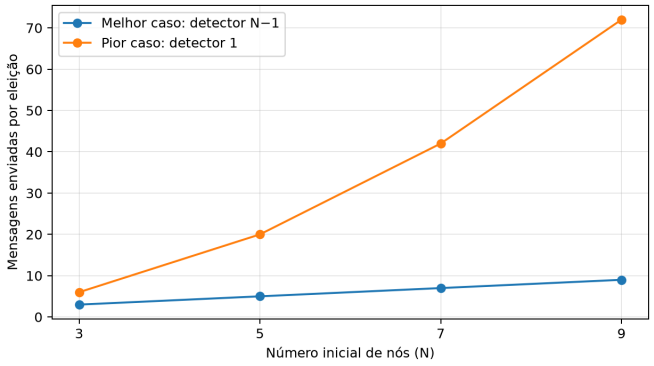


Figura 5. Mensagens por substituição do líder no simulador. Notificação imediata apenas em $N - 1$ (melhor caso) ou em 1 (pior). São contados todos os envios, inclusive ao nó morto. Dez repetições por ponto, sem variação observada.

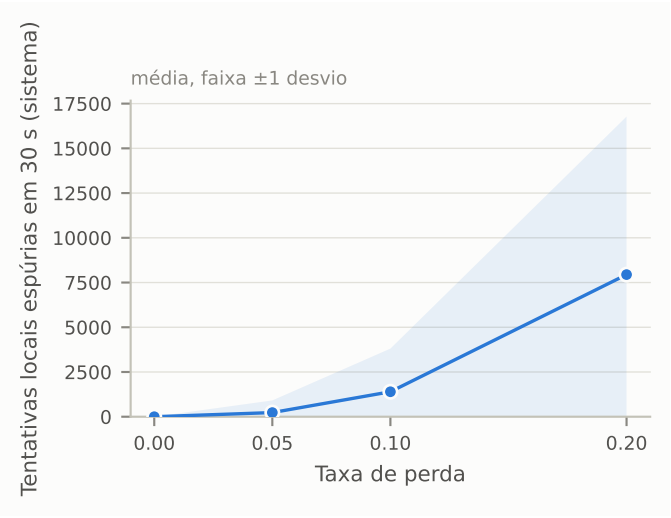


Figura 6. Tentativas locais espúrias por perda, com todos os nós ativos. Média de dez ensaios de 30 s; faixa indica ± 1 DP amostral. Inicialização excluída; perda aplicada também às mensagens de eleição.

caracterizam a implementação atual, sem modificações no algoritmo para reduzir as rajadas.

C. Discussão integrada

A identidade $L_f = L_d + L_e$ explica os resultados: reduzir a espera por ANSWER reduz o tempo de eleição, enquanto reduzir o timeout do detector antecipa seu início. Essa redução pode aumentar falsas suspeitas sob atraso ou perda. A varredura de perdas quantificou as tentativas espúrias resultantes com detector fixo, mas não comparou diferentes timeouts

Tabela VIII
TENTATIVAS LOCAIS ESPÚRIAS POR ENSAIO DE 30 S ($N = 5$). PERCENTIS COM INTERPOLAÇÃO LINEAR ENTRE DEZ SEMENTES.

Perda	n	**Média**	**DP**	**p50**	**p95**	**p99**
0%	10	0,0	0,0	0,0	0,0	0,0
5%	10	231,5	679,5	16,5	1207,9	1973,6
10%	10	1394,9	2412,3	65,0	5454,1	6833,2
20%	10	7943,5	8832,8	5998,0	22072,3	27283,2

nem o detector adaptativo. A eleição pode ter duração quase constante na faixa medida e ainda apresentar custo quadrático em mensagens; as duas métricas capturam aspectos distintos. Os experimentos Docker medem tempos com detecção concorrente, enquanto o simulador isola o iniciador para comparar os custos extremos.

VIII. LIMITAÇÕES

Os contêineres compartilham CPU, memória, kernel e rede do mesmo hospedeiro; não constituem domínios de falha independentes nem representam uma rede de longa distância. O instante do comando Docker não determina exatamente quando o processo deixou de executar. Os *timeouts* foram avaliados com apenas três repetições por nível, o que limita inferências sobre caudas. Na simulação, a notificação controlada do detector e a rede sem variabilidade isolam a complexidade, mas não reproduzem o escalonamento dos contêineres dentro de uma rede real. O Bully não usa quórum: partições podem produzir líderes simultâneos. A varredura de perdas quantifica tentativas espúrias com detector fixo; o benefício do detector adaptativo e a variação dos timeouts sob perda permanecem sem avaliação.

IX. CONCLUSÃO

A implementação integrou detecção de falhas e eleição de líder com transporte assíncrono, timeouts explícitos e logs verificáveis. Nos 78 ensaios Docker disponíveis, os sobreviventes convergiram para o maior ID vivo; o tempo de eleição foi dominado pela espera por ANSWER, enquanto a detecção e o disparo da falha explicaram a maior parcela da variabilidade do failover. Os 80 ensaios simulados confirmaram custo linear no melhor caso e quadrático no pior. Os 40 ensaios adicionais sob perda quantificaram as tentativas espúrias sem falhas reais e evidenciaram rajadas de reeleição.

Para prosseguimento dos experimentos podem ser realizados: os testes com o detector adaptativo para avaliar efeito da sua adição (esse caso se beneficiaria por utilizar uma simulação de conteinerês em uma rede real), variação dos timeout para melhor entendimento do comportamento e verificar o efeito dessas modificações sobre o algoritmo *Bully*.

CÓDIGO REUTILIZADO

Nenhum trecho de código de terceiros foi reutilizado na implementação dos algoritmos, do transporte ou do simulador. Usamos apenas a biblioteca padrão do Python (`asyncio`, `json`, `random`) nos nós; `pytest` nos testes; e `pandas` e `matplotlib` na análise. Foram utilizados agentes de desenvolvimento de código para auxiliar a planejar e implementar o projeto, especificamente Claude Code.

DIVISÃO DE TRABALHO

- **Guilherme Webster Chamoun** (RA 257111): fundação do projeto (configuração, mensagens, logs, transporte UDP com injeção de falhas, simulador), Docker/Compose/Makefile, detector de *heartbeat* e seu verificador, infraestrutura e experimentos do detector, redação.
- **Gabriel Cunha Marchetti** (RA 251055): eleição Bully, integração com o detector, métricas e experimentos do Bully, redação.

REFERÊNCIAS

[1] M. van Steen and A. S. Tanenbaum, *Distributed Systems*, 4th ed. distributed-systems.net, 2023.